

HelixCode

HelixCode

Die verteilte AI-Entwicklungsplattform, die Arbeit aufteilt, bewahrt und Ihren Fortschritt nie verliert.

Zusammenfassung

HelixCode ist eine unternehmensfähige, auf Go basierende verteilte AI-Entwicklungsplattform, die Entwicklungsaufgaben in intelligent unterteilte Aufgaben aufteilt und auf ein Netzwerk von SSH-gesteuerten Workern verteilt. Mit automatischer Checkpoint-Erstellung und Rollback-Funktion geht keine Arbeit verloren. Die Plattform vereint Multi-Provider-LLM-Integration, vollständige Entwicklungszyklus-Workflows sowie plattformübergreifende Bereitstellung hinter REST-, CLI-, TUI- und MCP-Schnittstellen.

Kurzbeschreibung

HelixCode ist eine verteilte AI-Entwicklungsplattform, geschrieben in Go. Sie unterteilt Arbeit in intelligente Aufgaben, die auf SSH-basierten Workernetzwerken ausgeführt werden, sichert Fortschritte durch automatische Checkpoint-Erstellung und Rollback, integriert mehrere LLM-Anbieter und steuert den gesamten Entwicklungszyklus über REST-, CLI-, TUI- und MCP-Schnittstellen.

Ausführliche Beschreibung

HelixCode ist eine unternehmensfähige verteilte AI-Entwicklungsplattform (`dev.helix.code`, MIT), die auf einem einfachen Versprechen basiert, das ihr Slogan wörtlich nimmt: die Arbeit aufteilen, sie bewahren und den Fortschritt nie verlieren. Die Plattform ist für intelligente Aufgabenverteilung, automatische Arbeitssicherung und plattformübergreifende Entwicklungsworkflows konzipiert. Sie ist in Go geschrieben – eine Sprache, die die für verteilte Systeme erforderliche

Nebenläufigkeit und Portabilität als Einzelbinary bietet – mit automatischer Checkpoint-Erstellung, Rollback-Funktion und Echtzeitüberwachung als grundlegende Primitive statt optionaler Erweiterungen.

Ihre Architektur legt eine REST- + WebSocket- + MCP-API-Oberfläche über einen Satz fokussierter Kernservices: JWT-Authentifizierung und Sitzungsverwaltung, SSH-basiertes Workermanagement mit Gesundheitsüberwachung, Aufgabenverwaltung mit Checkpoint-Erstellung und Abhängigkeitsmanagement, Projekt- und Workflowverwaltung sowie eine einheitliche LLM-Anbieterschicht – alles auf PostgreSQL persistiert, mit Redis als optionaler Koordinations- und Caching-Ebene. Verteilte Worker installieren sich automatisch im Netzwerk, sodass die Skalierung des Worker-Pools lediglich das Hinzufügen einer Maschine zum Server erfordert, statt manueller Provisionierung. Die Multi-Client-Schnittstellen umfassen CLI, Terminal-UI, REST und mobile Frameworks, sodass dieselbe Plattform sowohl aus einem Skript, einem Terminal als auch einer App erreichbar ist.

HelixCode steuert den gesamten Entwicklungszyklus von Anfang bis Ende: Planung, Bau, Testen und Refactoring-Workflows werden automatisch mit Abhängigkeitserkennung und Mehrsitzungs-Kontextverfolgung ausgeführt, sodass langlaufende Projekte ihren roten Faden über Unterbrechungen und Maschinengrenzen hinweg behalten. Die Plattform integriert mehrere LLM-Anbieter – llama.cpp, Ollama und OpenAI – hinter einer einheitlichen Schnittstelle und ergänzt dies durch hardwarebewusste Modellauswahl, die verfügbare CPU/GPU/Speicher erkennt und das Modell an die Maschine anpasst. Zudem unterstützt sie fortgeschrittene Reasoning-Strategien wie Chain-of-Thought und Tree-of-Thoughts für Probleme, die mehr als einen Durchlauf benötigen. Das Model Context Protocol ist über mehrere Transportwege für standardisierten Tool- und Kontextaustausch implementiert, und Multikanal-Benachrichtigungen (Slack, Discord, E-Mail, Telegram) halten Teams über den Fortschritt verteilter Arbeit auf dem Laufenden. Die Plattform ist für Linux, macOS, Windows, Aurora OS und SymphonyOS ausgelegt.

Warum wir es entwickelt haben

Verteilte und von AI unterstützte Entwicklung verliert typischerweise Kontext und Fortschritt, wenn Aufgaben auf verschiedene Maschinen aufgeteilt oder unterbrochen werden. HelixCode wurde entwickelt, um die Aufteilung von Aufgaben intelligent und die Sicherung von Arbeitsständen automatisch zu gestalten – sodass sich große Entwicklungsprojekte aufteilen,

über ein Netzwerk von Arbeitsknoten verteilen, zwischenspeichern und ohne Zustandsverlust fortsetzen oder zurücksetzen lassen.

Warum es ein Game-Changer ist

Es macht verteilte AI-Entwicklung *dauerhaft* – eine Fähigkeit, die bisher nie praktikabel war, wenn Teams diese Komponenten manuell zusammenfügten. Drei Funktionen, die normalerweise in drei separaten Tools stecken, werden zu einer einzigen Plattform: verteilter Rechenbetrieb (SSH-Arbeitsnetzwerke mit automatischer Installation und Gesundheitsüberwachung), AI-Entwicklungsunterstützung (Multi-Provider-LLMs mit Schlussfolgerungs- und Tool-Aufruffunktionen) und Automatisierung des gesamten Workflow-Lebenszyklus. Das verbindende Element ist die datenbankgestützte Zwischenspeicherung: Da Aufgabenstatus, Checkpoints und Abhängigkeiten in PostgreSQL persistiert werden, lässt sich ein Job, der über mehrere Maschinen und Sitzungen hinweg läuft, exakt an der Stelle zurücksetzen oder fortsetzen, an der er unterbrochen wurde. Unterbrechungen und aufgeteilte Arbeit sind nicht länger ein Grund für verlorenen Fortschritt, sondern werden zu einem routinemäßigen, wiederherstellbaren Ereignis.

Was innovativ ist

- **Arbeitsstandssicherung als grundlegendes Prinzip:** Automatische Zwischenspeicherung und Rücksetzung, angewandt auf *verteilte* Entwicklungsaufgaben, sodass Fortschritte Unterbrechungen und Maschinenausfälle überdauern, statt mit ihnen zu verschwinden.
- **Hardwarebewusste Modellauswahl**, die erkannte CPU-/GPU-/Speicherressourcen analysiert und jede Aufgabe einem Modell zuweist, das die Maschine tatsächlich effizient ausführen kann – ohne manuelle Anpassung pro Arbeitsknoten.
- **Eine Plattform, fünf Zugänge:** REST, WebSocket, CLI, TUI und MCP, wobei MCP selbst über mehrere Protokolle bereitgestellt wird, sodass sich Tools und Agenten unabhängig von ihrer Anbindung integrieren lassen.
- **Plattformübergreifende Reichweite**, die über das übliche Desktop-Trio hinausgeht und Aurora OS sowie SymphonyOS einbezieht – und so den Pool an Arbeitsknoten auf Plattformen erweitert, die die meisten Tools ignorieren.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **Kein Arbeitsverlust bei verteilten, unterbrechbaren Aufgaben.** Wird ein Job auf mehrere Maschinen aufgeteilt, bleiben bei einem Absturz oder einer Unterbrechung normalerweise alle laufenden Prozesse stecken. Wir haben die Aufgabe selbst als Träger von Checkpoints und Abhängigkeiten modelliert, die in PostgreSQL persistiert werden, sodass das System auf den letzten stabilen Zustand zurücksetzen oder von dort aus fortsetzen kann – eine Dauerhaftigkeit, die in der Datenebene verankert ist und nicht in flüchtigen In-Memory-Zuständen.
- **Verwaltung eines heterogenen Arbeitsknoten-Netzwerks.** Ein Netzwerk aus Linux-, macOS-, Windows-, Aurora- und SymphonyOS-Maschinen ist ein sich ständig änderndes Ziel in Bezug auf Verfügbarkeit und Einrichtung. Wir bewältigen dies mit einem dedizierten Arbeitsknoten-Pool-Dienst, der SSH-basierte Registrierung, automatische Installation auf neuen Knoten und kontinuierliche Gesundheitsüberwachung durchführt, sodass das Netzwerk auch bei wechselnden Maschinen überschaubar und steuerbar bleibt.
- **Heterogenität von Anbietern und Hardware.** LLM-Backends und die Maschinen, auf denen sie laufen, unterscheiden sich stark in ihren Fähigkeiten. Wir haben dies hinter einer einheitlichen LLM-Anbieter-Schnittstelle verborgen und mit Hardware-Erkennung (CPU/GPU/Speicher) kombiniert, die eine intelligente Modellauswahl steuert – sodass das passende Modell auf der richtigen Maschine landet, ohne dass der Aufrufer über beides nachdenken muss.

Technologie-Stack

- **Go (1.26+ Inner-Modul)** — ausgewählt, weil seine Goroutine-basierte Nebenläufigkeit und die Ausgabe als einzelnes Binary genau das bieten, was ein verteiltes Workersystem benötigt: kostengünstige Parallelität für die Orchestrierung und ein in sich geschlossenes Binary, das sich automatisch auf jedem Knoten installiert. Es enthält alle Kernservices sowie die CLI/Server-Binaries.
- **Gin (HTTP-Framework)** — ausgewählt für eine schnelle, minimalistische REST-Schicht mit geringem Overhead; es bedient die /api/v1-Schnittstelle (Authentifizierung, Worker, Aufgaben, Projekte), mit der jeder Client kommuniziert.
- **PostgreSQL 15+ (über pgx/v5)** — als dauerhafte Systemaufzeichnung gewählt, da Checkpointing und Rollback transaktionale Persistenz erfordern; es verwaltet das 11-Tabellen-Schema für verteilte Berechnungen (Benutzer,

Worker, Aufgaben, Projekte, Sitzungen, LLM-Anbieter, Benachrichtigungen), das die Arbeitsspeicherung ermöglicht.

- **Redis 7+ (optional, go-redis/v9)** — als optionale Caching- und Koordinationsschicht gewählt, die häufig genutzte Pfade beschleunigt, ohne eine harte Abhängigkeit darzustellen, sodass eine Minimalinstallation weiterhin allein mit PostgreSQL läuft.
- **SSH** — als Worker-Steuerungstransport ausgewählt, weil es bereits überall verfügbar und sicher ist; es steuert Worker-Registrierung, automatische Installation und Remote-Befehlsausführung im gesamten Pool, ohne dass zuvor ein spezieller Agent bereitgestellt werden muss.
- **Model Context Protocol (MCP)** — für den standardisierten Austausch von Tools und Kontexten gewählt, damit externe Tools und Agenten über ein offenes Protokoll integriert werden können; implementiert mit Multi-Transport-Unterstützung, um Clients dort abzuholen, wo sie sich verbinden.
- **LLM-Anbieter (llama.cpp, Ollama, OpenAI)** — ausgewählt, um sowohl lokale als auch gehostete Inferenz hinter einer einheitlichen Schnittstelle zu vereinen, sodass die hardwarebewusste Auswahl eine Aufgabe an ein lokales Modell oder ein gehostetes weiterleiten kann, ohne dass der Aufrufer den Unterschied bemerkt.

Status & Transparenzhinweise

- **Status: Beta.** Die README gibt einen „VOLLSTÄNDIG ABGESCHLOSSENEN / alle 5 Phasen“-Zustand an; diese Vollständigkeit ist projektintern deklariert und nicht unabhängig überprüft, daher wird der Status hier als Beta behandelt.
- Alle oben genannten Details stammen aus der README des Repositorys; Marketingformulierungen (Slogans) sind redaktionell und keine Quellmetriken.

Prioritätsstufe: Helix-primär.